Duke UNIVERSITY

Making Text-to-SQL Agents Worthy of Trust

An interactive lab for inspecting, testing, and governing the workflow behind a natural-language database answer.

Sung-Tse Wu · Duke University

Interactive artifact
text-to-sql-agent-lab-ids706.vercel.app
github.com/JayWu0512/Text-To-SQL-Agent-Lab

Try the Interactive Lab

Design question
How can a learner judge whether a Text-to-SQL agent deserves trust, rather than trusting SQL merely because it executes?

Abstract & motivation

Text-to-SQL systems can return fluent, executable queries that still use the wrong business definition, table grain, filter, or output format.

Natural-language question	→	SQL that runs
Does it answer the intended question?	?	Evidence is needed

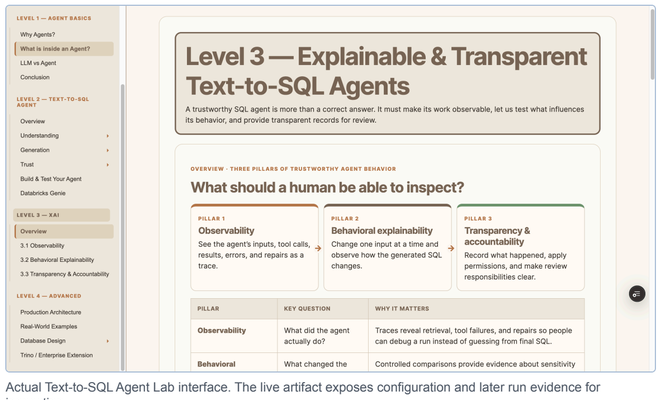
Common failure modes: valid syntax can still use the wrong business definition, aggregation grain, filter, or output format.

3	6	10×
XAI modules	agent components	runs per condition

System design

A browser-based course and live experiment lab using the Olist e-commerce schema. Learners configure components, run a question, and inspect evidence from the run.

1 Configure	2 Run	3 Inspect	4 Compare



Actual Text-to-SQL Agent Lab interface. The live artifact exposes configuration and later run evidence for inspection.

Inspectable evidence

- Retrieved definitions, schema documents, and verified examples
- A concise, student-visible SQL plan
- Generated SQL, read-only execution result, and evaluator outcome
- Validation feedback and bounded repair history

Question → retrieved evidence → SQL → execution → evaluation → repair record

Input	Question, selected agent components, model, and benchmark.
Grounding	Retrieved glossary, schema documents, examples, and value hints.
Action	Student-visible plan, generated SQL, validation decision, and execution rows.
Outcome	PASS/FAIL evaluation, accuracy, latency, and any repair attempts.

Methods & results: cumulative component study

Task. "For orders placed in 2018, calculate the total number of orders for each month, from earliest to latest." We held the model, evaluator, and read-only database fixed, then added generation components cumulatively. Each condition ran 10 times.

LLM only Baseline	0/10	No business semantics, schema linking, or example retrieval. Queries often returned month numbers or an extra year column.
+ Business semantics	0/10	Adds the glossary. Four runs added an unsupported delivered-order filter; remaining runs still used the wrong output shape.
+ Schema linking	0/10	Adds four retrieved schema documents per run. Every run found 10 months but produced a timestamp, month number, or extra year column.
+ Example retrieval	10/10	Adds four verified examples per run. This was the first successful configuration in this controlled sequence.
+ Query planning	10/10	Makes the intended filter, grouping, count, and ordering visible before SQL is generated.

Post-generation safeguards (not pass-rate conditions)

Guardrails
Validate generated SQL before execution. Block mutations, DDL, privileges, locks, and multiple statements; execute only read-only queries.

Repair
Use structured validation or database feedback to revise a failed query, with bounded retries and the original task preserved.

Evaluation
Compare generated results with the evaluator-owned ground truth, including rows, values, NULLs, duplicates, and required ordering.



Observed change: 0/10 → 10/10. Verified examples were associated with the first successful configuration in this cumulative experiment; this design does not isolate any component's independent effect.

Without verified examples
`EXTRACT(MONTH FROM timestamp)`
Wrong output shape: a month number or extra year column.

With verified examples
`DATE_TRUNC('month', timestamp)::date`
Correct output shape: the required monthly date grain.

Controlled comparison: question, model, evaluator, database, and read-only execution were fixed; the evaluator compared generated results against ground truth, including values and required ordering.

Key findings

Execution is not evidence of correctness. Trust requires inspectable evidence.

The lab makes retrieved context, generated SQL, execution, evaluation, and repair records available for review.

System performance
Does SQL produce the expected answer?

Explainability & observability
Can a learner inspect evidence, actions, and evaluation?

Human judgment
Can the learner identify errors and question assumptions?

Responsible AI framework

Observability
Review inputs, retrieved context, generated SQL, execution, evaluation, and repairs.

Behavioral explainability
Change one component and compare observable SQL and evaluation outcomes.

Transparency & accountability
Make policies and responsibilities explicit before high-risk actions occur.

Evidence exposed	Retrieved context, SQL, result, evaluation, repair history.
Human use	Debug a run, compare configurations, approve or stop risky actions.

Implementation safeguards

Read-only SQL
Only one SELECT, WITH, or EXPLAIN statement is accepted.

Mutation blocking
Write, DDL, privilege, and locking statements are rejected.

Bounded execution
Eight-second timeout and a 200-row display limit.

Result evaluation
Compares returned answers rather than SQL strings.

Limitations & future work

- Current evidence uses the Olist schema and one controlled benchmark question.
- Components were added cumulatively; independent effects and interactions were not isolated.
- Model outputs can vary by provider, model version, and run.
- The policy interface is educational; it is not production RBAC or an enterprise audit system.
- **Next:** evaluate across more questions, datasets, and models; add versioned experiment logs; and conduct a small user study comparing learners' error detection and trust calibration with and without the XAI interface.

What this poster claims
Measured, observable behavior for one fixed benchmark and configuration.

What it does not claim
Access to hidden reasoning, universal performance, or production-ready governance.

Related work

Text-to-SQL & evaluation
- Pourreza & Rafiei (2023), *DIN-SQL*
- Yu et al. (2018), *Spider*
- Li et al. (2023), *BIRD*

Explainability & governance
- Doshi-Velez & Kim (2017)
- NIST (2023), *AI RMF 1.0*
- Ribeiro et al. (2016), *LIME*